

# Stable Matching

Sofía Jiménez Venegas

Centro de Investigación en Computación, Instituto Politécnico Nacional, [sjimenezb25@cic.ipn.mx](mailto:sjimenezb25@cic.ipn.mx)

**Resumen**—Este reporte muestra el comportamiento asintótico del algoritmo de Gale-Shapley para resolver el Stable Matching Problem. Se implementó en un lenguaje de programación y se realizaron mediciones de tiempos de ejecución para comprobar su complejidad.

**Abstract**—This report shows the asymptotic behaviour of the Gale-Shapley algorithm for solving the Stable Matching Problem. It was implemented in a programming language and execution times were measured to verify its complexity.

## I. INTRODUCCIÓN

El stable matching problem se originó, en parte, en 1962, cuando David Gale y Lloyd Shapley, dos economistas matemáticos, plantearon la siguiente pregunta: ¿Se podría diseñar un proceso de admisión a la universidad o un proceso de selección de personal que se aplicara por sí solo? Gale y Shapley procedieron a desarrollar una solución algorítmica a este problema. [1]

En este caso, resolveremos el Stable Marriage Problem, en el cual tenemos dos grupos (hombres y mujeres), con la misma cantidad de personas. Cada miembro del grupo 1 tiene un ranking de preferencias de todos los miembros del grupo 2, y viceversa. El objetivo del algoritmo es emparejar a los miembros del grupo 1 con los del grupo 2, de manera que las parejas sean estables, esto significa que ninguna de las dos personas prefiera a otra que no sea su pareja asignada. [2]

## II. OBJETIVOS

- Implementar el algoritmo de Gale y Shapley para resolver el Stable Matching Problem en un lenguaje de programación.
- Generar a través de un programa M archivos de prueba para probar el algoritmo.
- Medir los tiempos de ejecución de los M archivos y graficarlos para comprobar la complejidad cuadrática del algoritmo.

## III. DESCRIPCIÓN DE ACTIVIDADES

### A. Metodología

Primero se analizó el pseudocódigo del algoritmo:  
 GALE-SHAPLEY (preference lists for men and women)  
 INITIALIZE S to empty matching.  
 WHILE (some man  $m$  is unmatched and hasn't proposed to every woman)  $w \leftarrow$  first woman on  $m$ 's list to whom  $m$  has not yet proposed.

```
IF (w is unmatched)
  Add pair  $m-w$  to matching S.
ELSE IF (w prefers  $m$  to her current partner  $m'$ )
  Remove pair  $m'-w$  from matching S. Add pair  $m-w$  to matching S.
ELSE
  w rejects m.
RETURN stable matching S. [3]
```

Cada “proponedor” puede proponerle a cada “receptor” como máximo una vez. Tenemos  $n$  hombres y  $n$  mujeres, suponiendo que los hombres proponen primero, en el peor de los casos, el hombre sería rechazado por todas las mujeres de su lista, menos por la última. Eso significa  $n$  propuestas por hombre. Como hay  $n$  hombres, el número máximo de propuestas es  $n^2$ .

En cada iteración del *while*, se hace una propuesta, como máximo hay  $n^2$  propuestas, todo lo demás dentro del *while* es  $O(1)$ , por lo tanto, la complejidad total del algoritmo es  $O(n^2)$ .

Se implementó el código en Python, para lo cual se utilizó diccionarios, siendo la llave del diccionario, la persona y la clave, una lista con el ranking de sus preferencias de las personas del otro grupo.

Dentro del bucle principal *while*, una vez que vamos realizando los emparejamientos, hacemos uso de otros diccionarios, para poder conocer en tiempo constante  $O(1)$ , quien esta emparejado con quien, y así se conserve la complejidad del algoritmo  $O(n^2)$ .

Una vez que se tiene funcionando el algoritmo, se realizó un código para generar M archivos de prueba, que tienen el siguiente formato:

```
2 w
w1 m1 m2
w2 m1 m2
m1 w1 w2
m2 w2 w1
```

Después generamos un código para medir:

- Tiempo de lectura del archivo
- Tiempo de ejecución del algoritmo (emparejamiento)
- Tiempo de impresión de resultados
- Tiempo total

Finalmente, graficamos el tiempo total de cada caso M.

### B. Herramientas

- Python
- IDE: Visual Studio Code
- GitHub

#### IV. RESULTADOS

La entrada de datos es de forma exponencial  $2^n$ , debido a la memoria del dispositivo con el que estamos trabajando, elegimos tener hasta  $2^{11}$  datos, estos fueron los tiempos medidos de cada archivo, de cuanto tarda en leerlo, el tiempo que le toma ejecutar el algoritmo y cuanto se tarda en imprimir los datos, así como el tiempo total.

TABLE I  
TIEMPOS DE EJECUCIÓN

| M  | n    | Lectura  | Emparejamiento | Impresión | Total    |
|----|------|----------|----------------|-----------|----------|
| 1  | 2    | 0.000676 | 0.000016       | 0.000001  | 0.000725 |
| 2  | 4    | 0.000503 | 0.000021       | 0.000002  | 0.000556 |
| 3  | 8    | 0.000561 | 0.000024       | 0.000002  | 0.000654 |
| 4  | 16   | 0.000704 | 0.000135       | 0.000004  | 0.000993 |
| 5  | 32   | 0.001292 | 0.000180       | 0.000005  | 0.001930 |
| 6  | 64   | 0.002491 | 0.000457       | 0.000011  | 0.004919 |
| 7  | 128  | 0.010879 | 0.001192       | 0.000019  | 0.021223 |
| 8  | 256  | 0.028944 | 0.006217       | 0.000052  | 0.069908 |
| 9  | 512  | 0.101181 | 0.003281       | 0.000040  | 0.183358 |
| 10 | 1024 | 0.312049 | 0.009997       | 0.000183  | 0.674616 |
| 11 | 2048 | 1.427272 | 0.039862       | 0.000478  | 2.985172 |

A continuación, se grafican los tiempos de ejecución obtenidos, conforme aumenta el tamaño de la entrada, en escala lineal (Figura 1) y escala logarítmica (Figura 2).

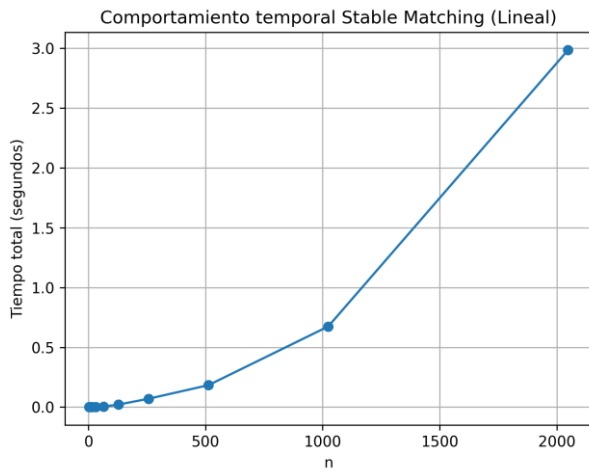


Fig. 1. Gráfica lineal

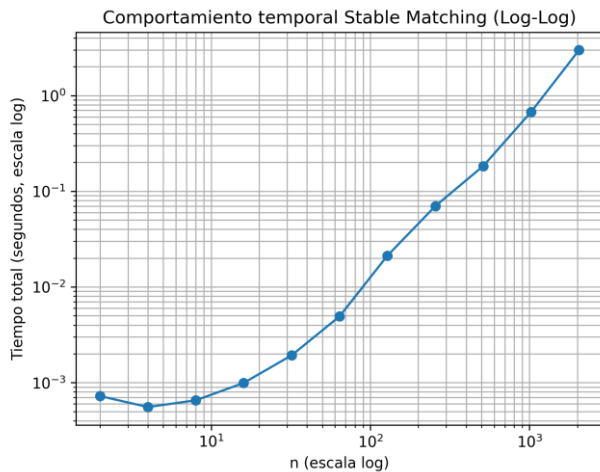


Fig. 2. Gráfica logarítmica

#### V. ANÁLISIS DE RESULTADOS

Podemos observar el crecimiento cuadrático de los datos, lo cual es el comportamiento esperado, ya que la complejidad del algoritmo es cuadrática  $O(n^2)$ .

Podemos ver para  $n < 500$ , no se observa el crecimiento de los datos con ese comportamiento, sin embargo, conforme  $n$  crece, la tendencia cuadrática se vuelve evidente.

También podemos ver que la mayor parte del tiempo de la ejecución del programa lo toma la lectura de los datos.

#### VI. CONCLUSIONES

El desarrollo de esta práctica nos ayudó a desarrollar el código del algoritmo de Gale-Shapley, así como comprobar de forma experimental la complejidad cuadrática del algoritmo.

Para ello, usamos herramientas, como los diccionarios de Python, para poder realizar búsquedas con complejidad constante  $O(1)$  y conservar la complejidad del algoritmo.

Debido al espacio de la memoria, no nos fue posible hacer crecer los datos de entrada tanto de forma exponencial, pero fueron los suficientes para poder observar el comportamiento del algoritmo.

#### REFERENCIAS

- [1] J. Kleinberg y É. Tardos, Algorithm Design, 1.ª ed. Boston, MA, EE. UU.: Pearson/Addison-Wesley, 2005.
- [2] GeeksforGeeks, "Stable Marriage Problem," GeeksforGeeks, 07-mar-2025. [En línea]. Disponible en: <https://www.geeksforgeeks.org/dsa/stable-marriage-problem/>. [Accedido: 27-feb-2026].
- [3] R. Acosta Bermejo, "Raúl Acosta Bermejo - CIC - IPN," Página personal académica. <https://www.cic.ipn.mx/views/researchers/sites/racostab/index.html> (accedido el 27 de febrero de 2026).